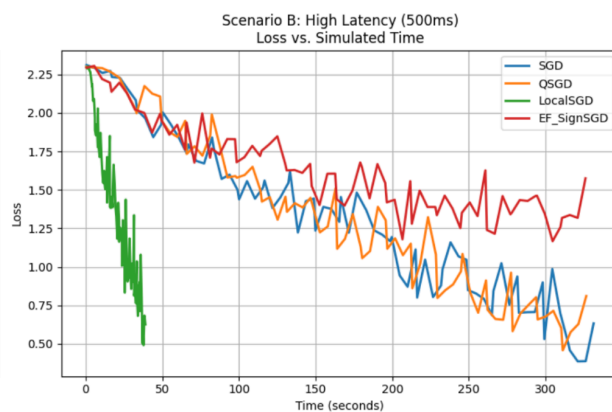
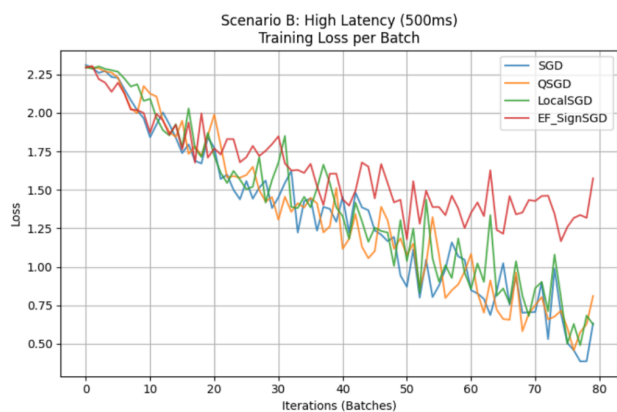
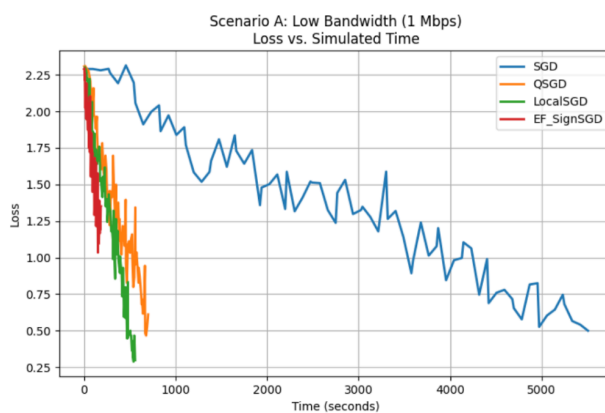
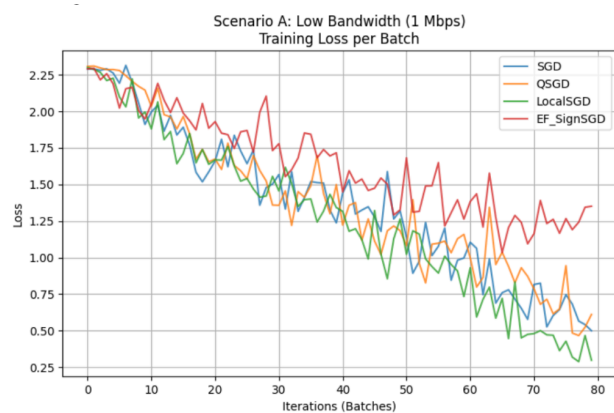
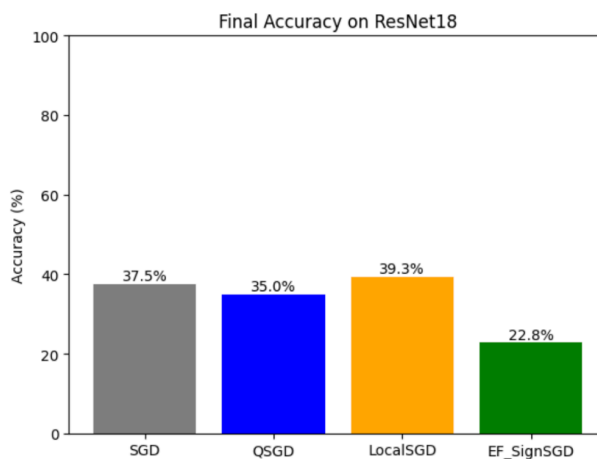
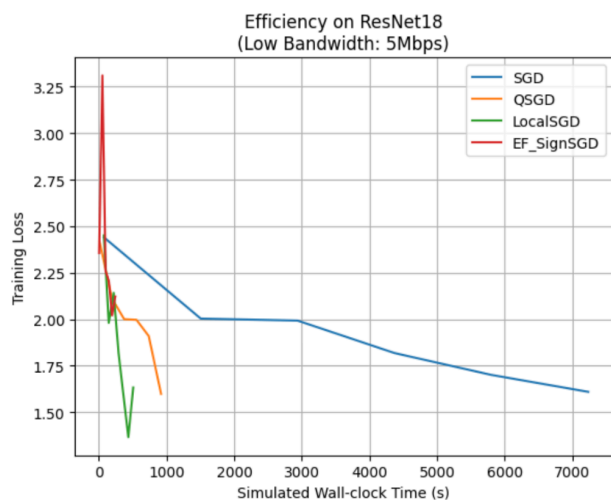


گزارش پروژه DMLS



ارائه

بخش ۱: مقدمه و زمینه تاریخی (Context & History)

هدف: پاسخ به سوال “چرا اینجا هستیم؟” و “چرا روش‌های قدیمی منسوخ شدند؟”

1. انفجار مدل‌ها: نمایش رشد مدل‌ها از ۲۰۱۲ (AlexNet) تا GPT-4.
 - مشکل: مدل روی یک GPU جا نمی‌شود یا خیلی کند است.
2. پارادایم **Data Parallelism** (توزیع داده):
 - توضیح: یک کپی از مدل روی تمام GPUها (WWW). داده‌ها تقسیم می‌شوند ($D_1, D_2, \dots, D_{1, D_2}$). همه گرادیان حساب می‌کنند و باید میانگین بگیرند.
 - پاسخ به سوال شما: بله، تمام این مقالات (QSGD, SignSGD, LocalSGD) روی **Data Parallelism** تمرکز دارند.
3. جنگ همگام (Sync) و ناهمگام (Async):
 - قدیم (۲۰۱۱-۲۰۱۵): از روش **Asynchronous SGD** (مثل Hogwild!) استفاده می‌شد. یعنی هر GPU کارش تمام شد، آپدیت می‌کند و منتظر بقیه نمی‌ماند.
 - چرا بود؟ چون شبکه کند بود و نمی‌خواستند معطل شوند.
 - چرا رفت؟ چون “Staleness” (بی‌بازی گرادیان) ایجاد می‌کرد. همگرایی مدل‌های جدید و دقیق را خراب می‌کرد.
 - اکنون (۲۰۱۶ به بعد): بازگشت به **Synchronous SGD** (همگام). همه منتظر هم می‌مانند تا میانگین دقیق گرفته شود.
 - گلوگاه جدید: حالا که همه منتظر می‌مانند، “زمان مخابره” ($T_{comm}T_{comm}$) گلوگاه اصلی است. هدف این پروژه کاهش این زمان است.

بخش ۲: استراتژی‌های کاهش ارتباطات (The Taxonomy)

هدف: دسته‌بندی راه‌حل‌ها.

برای کاهش ترافیک شبکه در SGD همگام، سه راه داریم:

1. **Quantization** (کوانتیزه‌سازی): کاهش دقت بیت‌ها (مثلاً ۳۲ بیت به ۴ بیت).
2. **Sparsification** (تُنک‌سازی): ارسال فقط بخشی از گرادیان‌ها (مثلاً فقط ۱۰٪ بزرگتر).
3. **Local SGD** (ارتباط کمتر): کاهش فرکانس ارتباط (مثلاً هر ۱۰ دور یکبار).

بخش ۳: کوانتیزه‌سازی نااریب (Unbiased Quantization)

مقاله کلیدی: Alistarh et al. (QSGD) - 2017

1. تعریف **Unbiased** (نااریب): یعنی اگر گرادیان را خراب (فشرده) می‌کنیم، به طور میانگین باید همان گرادیان اصلی باشد.
- ریاضی: $E[Q(g)] = g$
2. روش **QSGD**:

- از **Stochastic Rounding** (گرد کردن تصادفی) استفاده می‌کند.
- مثال: عدد 2.72.72.7 را با احتمال ۷۰٪ می‌کند 333 و با احتمال ۳۰٪ می‌کند 222.
- مزیت: همگرایی ریاضی تضمین شده است (چون میانگین درست است).
- عیب: واریانس (نویز) را زیاد می‌کند.

بخش ۴: فشرده‌سازی اریب و بازخورد خطا (Biased Compression & Error Feedback)

مقاله کلیدی: (Karimireddy et al. 2019) و SignSGD

1. چرا **Biased** شویم؟ چون می‌خواهیم فشرده‌سازی شدید داشته باشیم (مثلاً ۱ بیت). روش‌های Unbiased در ۱ بیت خطای وحشتناکی دارند.
2. روش **SignSGD**: فقط "علامت" گرادینان (± 1) را می‌فرستد.
- مشکل: این روش **Biased** (اریب) است. یعنی $E[\text{sign}(g)] \neq g$ است.
- اندازه بردار کلاً از دست می‌رود.
- نتیجه: اگر بدون اصلاح استفاده شود، مدل همگرا نمی‌شود.
3. **قهرمان داستان: (Error Feedback (EF**: پاسخ به سوال شما: EF یک روش فشرده‌سازی نیست؛ یک مکانیزم اصلاحی (Wrapper) است.
- منطق: خطای فشرده‌سازی امشب (ete_tet) را در حافظه نگه دار و فردا به گرادینان اضافه کن.
- تاثیر: Karimireddy ثابت کرد که EF تنها راهی است که باعث می‌شود SignSGD و Top-k همگرا شوند. نرخ همگرایی را از $O(1)O(1)O(1)$ (واگرا) به $O(1/T)O(1/\sqrt{T})O(1/T)$ (مثل SGD استاندارد) می‌رساند.

بخش ۵: روش Local SGD

مقاله کلیدی: (Stich 2019)

1. ایده: به جای فشرده‌سازی بسته، تعداد تماس‌ها را کم کن.
2. روش: هر Worker برای HHH مرحله روی دیتای خودش آپدیت می‌کند، بعد میانگین می‌گیرند.
3. تفاوت با فشرده‌سازی: اینجا پهنای باند (Bandwidth) مهم نیست، بلکه تأخیر (Latency) و Start-up Cost شبکه مهم است.
4. تحلیل Stich: نشان داد که اگر تعداد مراحل لوکال (HHH) خیلی زیاد نباشد، همگرایی دقیقاً مثل SGD کامل است.

بخش ۶: مبانی نظری و ریاضی (Theoretical Convergence)

پاسخ به سوالات ریاضی شما

1. **Convex vs. Non-Convex**: **Convex** (محدب): تابعی کاسه‌مانند که فقط یک دره (Min) دارد. اثباتش راحت است.

- **Non-Convex (غیرمحدب):** مثل رشته‌کوه (شبکه‌های عصبی). پر از دره‌های محلی. اثبات همگرایی برای این‌ها سخت و حیاتی است (چون دیپ لرنینگ Non-Convex است).
- 2. **معیار همگرایی (Convergence Metric):**
 - برای Non-Convex، نمی‌توانیم بگوییم چقدر به "بهترین جواب" نزدیکیم (چون نمی‌دانیم کجاست).
 - پس می‌سنجیم که "چقدر شیب (گرادیان) صفر شده است". اگر گرادیان صفر شود، یعنی در یک چاله گیر کرده‌ایم (که خوب است).
- فرمول: $E[\|\nabla f(x)\|^2] \leq \epsilon$
- 3. **Upper Bound vs. Lower Bound:**
 - **Lower Bound (کران پایین):** یعنی "خداوند متعال" می‌گوید هیچ الگوریتمی در این شرایط نمی‌تواند سریع‌تر از این سرعت همگرا شود (محدودیت ذاتی مسئله).
 - **Upper Bound (کران بالا):** سرعتی که الگوریتم پیشنهادی ما تضمین می‌کند.
 - هنر مقاله: نویسنده سعی می‌کند Upper Bound روش خودش را به Lower Bound مسئله نزدیک کند تا بگوید روش من "Optimal" است.

بخش ۷: لیست ۲۰ مقاله کلیدی (Roadmap)

این لیست را دقیقاً به همین ترتیب در اسلایدها یا رفرنس‌ها استفاده کنید. از کلاسیک به مدرن:

الف) پایه‌های SGD توزیع‌شده (Foundations)

1. **Dean et al. (2012): Large Scale Distributed Deep Networks** (معرفی Parameter Server و Async SGD).
2. **Li et al. (2014): Scaling Distributed Machine Learning with the Parameter Server**.
3. **Seide et al. (2014): 1-Bit Stochastic Gradient Descent** (اولین ایده ۱ بیتی + EF اولیه).
4. **Alistarh et al. (2017): QSGD: Communication-Efficient SGD** (تئوری Unbiased - بسیار مهم).
5. **Wen et al. (2017): TernGrad: Ternary Gradients** ... (گرادیان سه حالته).
6. **Bernstein et al. (2018): signSGD: Compressed Optimisation for Non-Convex Problems** (فشرده‌سازی علامت).

ج) تنک‌سازی (Sparsification)

7. **Aji & Heafield (2017): Sparse Communication** ... (ایده Top-k: فقط بزرگترین‌ها را بفرست).
8. **Lin et al. (2018): Deep Gradient Compression (DGC)** (تکمیل Top-k با مومنتوم).

د) بازخورد خطا و اصلاح (Error Feedback)

9. **Stich et al. (2018): Sparsified SGD with Memory** (تحلیل ریاضی EF برای حالت Convex).
10. **Karimireddy et al. (2019): Error Feedback Fixes SignSGD** ... (تحلیل ریاضی EF برای Non-Convex - بسیار مهم).

Local SGD (ه)

11. *McMahan et al. (2017): Communication-Efficient Learning... (FedAvg)* (پایه Local SGD در Federated Learning).

12. *Stich (2019): Local SGD Converges Fast and Communicates Little* (تحلیل همگرایی Local SGD - بسیار مهم).

13. *Lin et al. (2020): Don't Use Large Mini-Batches, Use Local SGD*.

(و) روش‌های پیشرفته و مدرن (Advanced)

14. *Vogels et al. (2019): PowerSGD* (استفاده از تجزیه ماتریس Low-Rank).

15. *Tang et al. (2021): 1-bit Adam* (ترکیب فشرده‌سازی با Adam).

16. *Safaryan et al. (2021): Uncertainty Principle for Communication Compression*.

17. *Richtarik et al. (2021): EF21: A New, Simpler, Theoretically Better... Error Feedback* (نسخه جدیدتر EF).

(ز) مقالات مروری (Survey)

18. *Xu et al. (2020): A Survey on Communication-Efficient Distributed Training*.

19. *Tang et al. (2020): Communication-Efficient Distributed Deep Learning: A Comprehensive Survey*.

20. *Liu et al. (2020): A Distributed SGD Survey* (تمرکز روی همگرایی).

تحلیل و ارزیابی و مقایسه**بخش ۸: طراحی آزمایشات و تحلیل مقایسه‌ای (Project Design & Comparative Analysis)**

هدف: پاسخ به سوال "کدام روش در چه شرایطی برنده است؟"

شما باید یک ماتریس ارزیابی تعریف کنید. ادعای اصلی این پروژه این است که "هیچ روشی مطلقاً بهترین نیست؛ برنده به شرایط شبکه و مدل بستگی دارد."

۱. سناریوهای شبکه (Network Conditions)

ما دو پارامتر اصلی شبکه داریم: پهنای باند (Bandwidth) و تأخیر (Latency).

- سناریوی A: پهنای باند کم (Low Bandwidth)
- مثال: آموزش روی اینترنت خانگی یا Federated Learning (اینترنت موبایل).
- چالش: ارسال گرادینانهای بزرگ (مثلاً ۱۰۰ مگابایت) خیلی طول می‌کشد.
- تحلیل روش‌ها:
- (QSGD (Alistarh): عملکرد عالی. با کاهش بیت‌ها حجم را کم می‌کند.
- (SignSGD (Karimireddy): بهترین عملکرد (فشرده‌سازی ۳۲ برابری).
- (Local SGD (Stich): خوب است، اما چون در نهایت باید مدل کامل را بفرستد، ممکن است در لحظه همگام‌سازی گلوگاه شود.
- برنده احتمالی: SignSGD + EF (چون بیت کمتری مصرف می‌کند).
- سناریوی B: تأخیر بالا (High Latency)
- مثال: سرورها در قاره‌های مختلف (Geo-distributed). زمان "Handshake" شبکه زیاد است، حتی اگر حجم داده کم باشد.
- چالش: منتظر ماندن برای پاسخ بقیه (Round-trip time) کشنده است.
- تحلیل روش‌ها:
- QSGD & SignSGD: عملکرد ضعیف. این روش‌ها در هر دور (Iteration) نیاز به ارتباط دارند. اگر پینگ شبکه ۱۰۰ میلی‌ثانیه باشد، در هر دور این زمان تلف می‌شود.
- (Local SGD (Stich): برنده مطلق. مقاله Stich نشان می‌دهد که با انجام HHH مرحله لوکال، تعداد کل ارتباطات را HHH برابر کاهش می‌دهیم. این یعنی فرار از تأخیر شبکه.

۲. تأثیر اندازه مدل (Model Size Impact)

- مدل‌های کوچک (مثل CNNهای ساده یا ResNet-18):
- محاسبات (Computation) سریع انجام می‌شود. گلوگاه اصلی همان شبکه است.
- نتیجه: فشرده‌سازی گرادینان تأثیر چشمگیری روی زمان کل می‌گذارد.
- مدل‌های بزرگ (مثل GPT-Tiny یا VGG):
- محاسبات سنگین است. نسبت Computation/Communication بالاست.
- نکته ظریف: در مدل‌های خیلی بزرگ، Local SGD ممکن است باعث واگرایی شود (چون مدل‌ها در لوکال خیلی از هم دور می‌شوند). اینجا Error Feedback (مقاله Karimireddy) اهمیت پیدا می‌کند تا همگرایی را با وجود فشرده‌سازی حفظ کند.

۳. معیارهای ارزیابی (Metrics)

برای اینکه پروژه شما علمی باشد، نباید فقط "دقت نهایی" را گزارش کنید. باید نمودارهای زیر را رسم کنید:

1. Loss vs. Communication Volume (دقت در برابر حجم دیتا):

- محور افقی: مگابایت‌های ارسال شده.
- محور عمودی: خطای مدل (Loss).
- هدف: نشان دهید QSGD و SignSGD خیلی سریع‌تر (با دیتای کمتر) به خطای پایین می‌رسند.

2. Loss vs. Wall-clock Time (دقت در برابر زمان واقعی):

- محور افقی: زمان (ثانیه).
- محور عمودی: خطای مدل.
- هدف: اینجا Local SGD در محیط‌های High Latency باید برنده شود.
- نکته پیاده‌سازی: اگر کلاستر واقعی ندارید، می‌توانید در کد خود "تأخیر مصنوعی" (Artificial Delay) تزریق کنید (`time.sleep` بر اساس حجم گرادیان).

بخش ۹: مسیرهای پیشنهادی پروژه (Project Paths)

کاربر گرامی، بسته به علاقه و توانایی خود، یکی از این سه مسیر را برای پروژه نهایی انتخاب کنید:

مسیر الف: تحلیل همگرایی نظری (Theoretical Analysis Focus)

مناسب برای: کسانی که ریاضیات قوی دارند.

- کار: اثبات یک قضیه (Theorem) جدید یا بسط قضایای موجود.
- ایده: مقاله Stich (Local SGD) همگرایی را برای توابع Non-Convex ثابت کرد، اما فرض کرد گرادیان‌ها Bounded هستند. مقاله Karimireddy (EF) همگرایی SignSGD را ثابت کرد.
- نوآوری شما: ترکیب این دو. "تحلیل همگرایی Local SignSGD with Error Feedback".
- یعنی هم Local آپدیت کنیم و هم وقتی می‌فرستیم، فقط علامت را بفرستیم و خطا را لوکال نگه داریم.
- آیا اثبات ریاضی آن همگرا می‌شود؟ (یک چالش جذاب آکادمیک).

مسیر ب: ارزیابی تجربی روی کلاستر (Empirical Evaluation)

مناسب برای: مهندسی سیستم و DevOps.

- کار: پیاده‌سازی ۴ الگوریتم (SGD, QSGD, SignSGD+EF, Local SGD) در PyTorch Distributed.
- محیط اجرا: استفاده از Google Colab (چند GPU) یا AWS یا سرور دانشگاه.
- خروجی: تولید نمودارهای بخش ۸ با دیتای واقعی (مثلاً روی CIFAR-10). نشان دهید که در شبکه کند، SignSGD پنج برابر سریع‌تر از SGD معمولی است.

مسیر ج: شبیه‌سازی و مقایسه (Simulation Framework)

مناسب برای: برنامه‌نویسی پایتون بدون درگیری با پیچیدگی‌های Distributed واقعی.

- کار: نوشتن یک شبیه‌ساز (Simulator) ساده.
- روش: به جای شبکه واقعی، یک کلاس Network بسازید که گرادیان را می‌گیرد، بر اساس فرمول $\text{Time} = \text{Latency} + (\text{Size} / \text{Bandwidth})$ زمان را محاسبه می‌کند و گرادیان را (با نویز کوانتیزه) برمی‌گرداند.

- **مزیت:** می‌توانید به راحتی نمودار بکشید که “اگر پهنای باند نصف شود، QSGD چقدر جلو می‌افتد؟”. این برای یک ارائه کلاسی یا پایان‌نامه بسیار تمیز و قابل فهم است.

بعضی جزییات از الگوریتم EF-signSGD

1. Vanilla SignSGD (The original)

As defined on **Page 1** of the paper, the update rule for standard SignSGD is:

$$x_{t+1} := x_t - \gamma \text{sign}(g_t)$$

- **Scale:** There is **no adaptive scale**. It treats every gradient the same, whether the value is \$0.0001\$ or \$100.0\$. It only cares if the number is positive or negative.
- **The Problem:** This is exactly why it often fails to converge. It "forgets" how large the gradient was, which is crucial for knowing how close you are to the optimum.

2. EF-SignSGD (The "Fixed" version)

The authors introduce the scale as part of the "fix" to make SignSGD work like regular SGD. In **Algorithm 1 (Page 2)**, the update is:

$$\Delta_t := \left(\frac{p_t}{p_{t-1}} \right)^d \text{sign}(p_t)$$

- **The Scale:** The term $\frac{\|p_t\|_1}{d}$ is the scale. It represents the **average magnitude** of the gradient components.
- **Why it was added:** The paper explicitly states: "*problems of SIGN SGD can be fixed by i) scaling the signed vector by the norm of the gradient to ensure the magnitude is not forgotten.*"

Summary Table

Feature	Vanilla SignSGD	EF-SignSGD
Formula	$\text{sign}(g_t)$	$\text{scale} \times \text{sign}(p_t)$
Magnitude	Lost (all updates are size 1)	Preserved (via the scale factor)
Error Feedback	No	Yes
Convergence	Can diverge on simple problems	Converges as fast as full SGD

The Core Idea: Error Feedback (EF)

When you use **SignSGD**, you throw away almost all the information about the gradient (you only keep the "+" or "-" sign). If the gradient was \$0.9\$, you send \$+1\$. If it was \$0.001\$, you also send \$+1\$. That "lost" information (the \$0.899\$ you threw away) is the **Error**.

Error Feedback says: "Don't throw that \$0.899\$ away. Save it in a local buffer and add it to the next gradient you calculate."

Step-by-Step Algorithm (English)

Imagine you are a worker (a computer) in a cluster:

1. **Calculate Gradient:** Compute your local stochastic gradient g_t based on your data.
2. **Add Accumulated Error:** Take the error you saved from the *previous* step (e_t) and add it to your current gradient. Let's call this sum the "Corrected Gradient" (p_t):

$$p_t = g_t + e_t$$
3. **Compress (Sign + Scale):** Instead of sending the full p_t , you compress it. You take its sign, but to keep the scale realistic, you multiply it by the average magnitude (the

L_1 norm):

$$\Delta_t = \text{scale} \times \text{sign}(p_t)$$

(This Δ_t is what you actually send to the server/other workers).

4. **Update Error Buffer:** Calculate how much information you just lost by compressing and save it for next time:

$$e_{t+1} = p_t - \Delta_t$$
5. **Update Model:** Move your local model weights using the compressed update:

$$x_{t+1} = x_t - \text{learning_rate} \times \Delta_t$$

A Simple Numerical Example

Let's say your learning rate is 1 and you only have 1 dimension for simplicity.

- **Step 1:** Your gradient is $g_1 = 0.8$. Your error $e_1 = 0$.
 - $p_1 = 0.8 + 0 = 0.8$.
 - You compress it: $\text{sign}(0.8) = +1$. You send +1.
 - **New Error:** $e_2 = 0.8 - 1 = -0.2$. (You "over-shot" by 0.2).
- **Step 2:** Your next gradient is $g_2 = 0.5$.
 - $p_2 = 0.5 + (-0.2) = 0.3$. (The error corrected the gradient!).
 - You compress it: $\text{sign}(0.3) = +1$. You send +1.
 - **New Error:** $e_3 = 0.3 - 1 = -0.7$.

Why does this fix the problem?

Without Error Feedback, the small values (0.1, 0.2) might keep getting rounded up to 1.0 every single time, making the model diverge. With EF, the "over-shooting" is subtracted from the next step, ensuring that **on average**, the direction of your updates stays true to the real gradients.

ادامه تحلیل ها:

۱. چرا Local SGD برای HPC و شبکه سریع (Infiniband)؟

در محیط‌های HPC، مشکل اصلی حجم داده (Bandwidth) نیست، بلکه سرریز همگام‌سازی (Synchronization Delay) است.

• مستند در مقاله Stich (صفحه ۱ و ۲):

او در مقدمه مستقیماً اشاره می‌کند که حتی در شبکه‌های پرسرعت، Mini-batch SGD به دلیل «تاخیرهای شبکه» (Network Delays) دچار کندی می‌شود. او می‌گوید:

The scheme... often suffers from large network delays... To overcome this, recent works "propose to reduce the communication frequency"

- **تحلیل:** در شبکه Infiniband، فرستادن ۱ مگابایت یا ۱۰ مگابایت تفاوت زمان ناچیزی دارد، اما «منتظر ماندن» برای اینکه همه ۱۰۰ کارگر اعلام آمادگی کنند (Synchronization Barrier) زمان‌بر است. Local SGD با کاهش تعداد دفعات این انتظار، سرعت را بالا می‌برد.

۲. چرا QSGD برای اینترنت عمومی و Federated Learning؟

در این محیط‌ها (مثل آموزش روی گوشی‌های موبایل)، مشکل اصلی **پهنای باند بسیار محدود** است. شما نمی‌توانید مدام مدل‌های چند صد مگابایتی را آپلود کنید.

- **مستند در مقاله Alistarh (بخش Abstract و صفحه ۲):**
نویسنده صراحتاً می‌گوید هدف QSGD کاهش هزینه پهنای باند (Bandwidth cost) است. او اشاره می‌کند که QSGD به شما اجازه می‌دهد تعداد بیت‌های ارسالی را بر اساس محدودیت شبکه تنظیم کنید.
"QSGD allows the user to smoothly trade off communication bandwidth and convergence time".
- **تحلیل:** در اینترنت عمومی، حجم داده ارسالی مستقیماً با هزینه و زمان رابطه خطی دارد. پس فشردسازی (Quantization) اولویت اول است.

۳. چرا Error Feedback برای شرایط سخت (مثل SignSGD)؟

- **مستند در مقاله Karimireddy (صفحه ۱):**
او اشاره می‌کند که در محیط‌های توزیع‌شده بزرگ، ما مجبوریم از فشردسازی‌های بسیار شدید (مثل فقط ارسال علامت گرادیان) استفاده کنیم تا گلوگاه ارتباطی حل شود، اما این کار باعث واگرایی می‌شود.
- **تحلیل:** او ثابت می‌کند که با Error Feedback، می‌توانیم "Gradient compression for free" داشته باشیم؛ یعنی بدون افت دقت، حجم را کم کنیم. این برای محیط‌هایی که شبکه بسیار ضعیفی دارند (مثل Federated Learning) حیاتی است.

جمع‌بندی بر اساس متن مقالات:

ارجاع	راهکار پیشنهادی مقالات	گلوگاه (Bottleneck)	شرایط شبکه
Stich (2019)	Local SGD (کاهش فرکانس)	دفعات ارتباط (Latency)	خوشه HPC / سریع
Alistarh (2017) / Karimireddy (2019)	QSGD / EF-SignSGD (کاهش حجم)	حجم داده (Bandwidth)	اینترنت / موبایل

درباره انواع QSGD

۱. تنظیم تعداد سطوح کوانتیزه‌سازی (\$s\$)

در QSGD، شما یک پارامتر به نام \$s\$ دارید که تعداد سطوح (levels) فشرده‌سازی را تعیین می‌کند.

- اگر \$s\$ را کوچک انتخاب کنید (مثلاً ۲ سطح)، گرادینت بسیار فشرده می‌شود (کم‌حجم) اما نویز آن زیاد می‌شود.
 - اگر \$s\$ را بزرگ انتخاب کنید، دقت بالاتر می‌رود اما حجم داده ارسالی بیشتر می‌شود.
- این یعنی QSGD یک الگوریتم واحد نیست، بلکه طیفی از الگوریتم‌هاست که بین "پهنای باند" (Bandwidth) و "زمان همگرایی" (Convergence Time) تعادل برقرار می‌کنند.

۲. انعطاف‌پذیری در کدگذاری (Encoding)

مقاله چندین روش برای بسته‌بندی بیت‌ها پیشنهاد می‌دهد:

- QSGD ساده: که در آن فشرده‌سازی انجام می‌شود.
- QSGD با کدگذاری الیاس (Elias Coding): که یک روش کدگذاری بهینه برای کاهش بیشتر حجم داده است تا به محدودیت‌های تئوریک (Information-theoretic lower bounds) نزدیک شود.

۳. پشتیبانی از سناریوهای مختلف

این خانواده از الگوریتم‌ها برای حالت‌های مختلف طراحی شده است:

- حالت محدب (Convex): برای مسائل بهینه‌سازی ساده‌تر.
- حالت غیرمحدب (Non-convex): برای شبکه‌های عصبی عمیق (Deep Learning).
- حالت ناهمگام (Asynchronous): زمانی که کارگرها (Workers) منتظر یکدیگر نمی‌مانند.

چرا این موضوع مهم است؟

در مقاله Alistarh (صفحه ۲)، نویسندگان تأکید می‌کنند که برخلاف روش‌های فشرده‌سازی قبلی (مثل 1BitSGD) که فقط یک حالت خاص بودند و گاهی واگرا می‌شدند، QSGD یک چارچوب ریاضی ارائه می‌دهد که تضمین می‌کند در تمام این حالت‌ها (اگر پارامترها درست انتخاب شوند) مدل حتماً همگرا خواهد شد.

روش‌ها (Sparsification و Quantization) عمدتاً برای حل چالش‌های (Data Parallelism (DP ابداع شدند، اما امروزه در ابعاد بزرگتر برای Model Parallelism هم حیاتی هستند.

بیا بید دلیل فنی این موضوع را بررسی کنیم:

۱. چرا این روش‌ها در اصل برای (Data Parallelism (DP هستند؟

در Data Parallelism، ما مدل را روی مثلاً ۸ عدد GPU کپی می‌کنیم. هر GPU بخشی از داده‌ها را می‌بیند و یک "گرادینت" (Gradient) محاسبه می‌کند.

- مشکل گلوگاه (Bottleneck): در انتهای هر مرحله، تمام این ۸ مدل باید با هم صحبت کنند تا گرادینت‌هایشان را میانگین بگیرند (عملیات All-Reduce).

- **حجم داده:** اگر مدل شما مثلاً ۷ میلیارد پارامتر داشته باشد (مثل Llama-2-7B)، هر GPU باید حدود ۲۸ گیگابایت دیتا (در حالت float32) را در شبکه جابه‌جا کند. این کار باعث می‌شود GPUها بیکار بمانند تا انتقال دیتا در شبکه (Network Bandwidth) تمام شود.

راه حل‌ها اینجا وارد می‌شوند:

- **Quantization:** به جای فرستادن گرادیان‌ها با دقت ۳۲ بیتی، آن‌ها را به ۸ بیت یا حتی ۴ بیت تبدیل می‌کنیم. حجم ترافیک شبکه ۴ تا ۸ برابر کم می‌شود.
- **Sparsification:** ما متوجه شدیم که ۹۹٪ گرادیان‌ها در هر مرحله تغییرات بسیار ناچیزی دارند. پس فقط ۱٪ از مهم‌ترین گرادیان‌ها را ارسال می‌کنیم (Sparse SGD). این کار پهنای باند مورد نیاز را به شدت کاهش می‌دهد.

۲. آیا در (Model Parallelism (MP هم کاربرد دارند؟

بله، اما با یک هدف متفاوت. در MP (به‌ویژه Tensor Parallelism)، اجزای یک لایه بین GPUها تقسیم شده‌اند.

- **در زمان آموزش:** این روش‌ها کمتر برای جابه‌جایی گرادیان استفاده می‌شوند و بیشتر برای کاهش حافظه مصرفی (VRAM) به کار می‌روند تا بتوان مدل‌های بزرگتر را در تعداد GPU کمتر جا داد.
- **در زمان استنتاج (Inference):** اینجا جایی است که Quantization (مثل متدهای AWQ یا GPTQ) یا پادشاهی می‌کند تا مدل‌های عظیم روی سخت‌افزارهای معمولی اجرا شوند.

۳. تفاوت کلیدی در اعمال روی SGD

وقتی صحبت از SGD (بهینه‌ساز) می‌شود، تمرکز مستقیماً روی "آپدیت‌ها" است:

هدف اصلی	کاربرد در SGD	روش
کاهش حجم بسته‌های ارسالی در شبکه.	تبدیل float32 به int8 یا FP8 در زمان ارسال گرادیان.	Quantization
کاهش تعداد دفعات ارتباط (Communication Frequency).	ارسال فقط گرادیان‌های بزرگتر از یک آستانه خاص (Threshold).	Sparsification

یک نکته حرفه‌ای: Local SGD

فراتر از این دو روش، مدتی به نام Local SGD هم وجود دارد که مخصوص DP است. در این روش، هر GPU برای مثلاً ۱۰ مرحله به تنهایی SGD را انجام می‌دهد و فقط هر ۱۰ مرحله یک‌بار با بقیه هماهنگ می‌شود. این هم نوعی تلاش برای فرار از محدودیت‌های ارتباطی در Data Parallelism است.

روش Sparsification

ایده اصلی پشت این که چطور دقت مدل با وجود **Quantization** (کم کردن دقت اعداد) یا **Sparsification** (حذف ۹۹٪ گرادیان‌ها) پایین نمی‌آید، تکنیکی به نام **Error Compensation** یا **Error Feedback** است.

این تکنیک مثل یک "دفترچه یادداشت" برای اشتباهات عمل می‌کند.

۱. مکانیزم جبران خطا (Error Compensation)

فرض کن می‌خواهی عدد ۳.۷ را ارسال کنی، اما سیستم کوانتیزاسیون تو فقط اعداد صحیح (Integer) می‌پذیرد.

- تو عدد ۳ را ارسال می‌کنی.
- خطای تو چقدر است؟ ۰.۷.
- در گام بعدی (Step بعدی SGD)، این ۰.۷ را به گرادیان جدید اضافه می‌کنی. اگر گرادیان بعدی ۲.۵ باشد، حالا مجموع می‌شود ۳.۲.
- حالا عدد ۳ را ارسال می‌کنی و خطای جدید (۰.۲) را دوباره برای گام بعد ذخیره می‌کنی.

نتیجه: با این روش، هیچ اطلاعاتی "دور ریخته" نمی‌شود، بلکه فقط "به تاخیر" می‌افتد. در نهایت میانگین گرادیان‌های ارسالی با مقدار واقعی برابر می‌شود.

۲. چرا در Sparsification معجزه می‌کند؟

در روش‌های Sparse (مثل **Top-k Selection**)، ما فقط بزرگترین تغییرات را می‌فرستیم. بدون جبران خطا، مدل هرگز همگرا نمی‌شود چون تغییرات کوچک اما مهم دائمی حذف می‌شوند. اما با **Error Feedback**، این تغییرات کوچک آنقدر در "بافر خطا" روی هم جمع می‌شوند تا بالاخره بزرگ شده و در یک مرحله ارسال شوند.

۳. الگوریتم‌های معروف (برای مطالعه عمیق‌تر)

اگر بخواهی این‌ها را در کد یا مقالات دنبال کنی، این نام‌ها کلیدی هستند:

- **DoubleSqueeze**: ترکیبی از کوانتیزاسیون و اسپارسیفیکیشن با جبران خطای دوطرفه.
- **1-bit SGD**: روشی از مایکروسافت که گرادیان‌ها را فقط به مثبت یا منفی (۱ بیت) تبدیل می‌کند و با **Error Feedback** دقت را حفظ می‌کند.
- **Deep Gradient Compression (DGC)**: روشی که نرخ فشرده‌سازی را تا ۶۰۰ برابر بالا می‌برد بدون اینکه دقت مدل ImageNet یا مدل‌های زبانی کم شود.

۴. یک چالش جدی: کند شدن محاسبات (Overhead)

یک نکته که معمولاً در تئوری گفته نمی‌شود: گاهی اوقات زمانی که GPU صرف می‌کند تا بفهمد کدام ۱٪ گرادیان‌ها "بزرگتر" هستند (Sparsification) یا عملیات فشرده‌سازی را انجام دهد، بیشتر از زمانی است که برای فرستادن کل دیتا در یک شبکه پرسرعت (مثل NVLink) نیاز دارد!

به همین دلیل:

- در شبکه‌های کند (مثل اینترنت یا اترنت معمولی): این روش‌ها معجزه می‌کنند.
- در ابررایانه‌ها با NVLink: معمولاً از این روش‌ها استفاده نمی‌شود چون سرعت جابه‌جایی دیتا آنقدر بالاست که فشرده‌سازی فقط وقت تلف کردن است.

روش Top-k Selection with Momentum

روش **Top-k Selection with Momentum** یک تکنیک پیشرفته در فشرده‌سازی گرادیان (Gradient Compression) برای آموزش توزیع‌شده (Distributed Training) است. این روش ترکیبی از دو ایده هوشمندانه است: **Sparsification** (حذف گرادیان‌های کوچک) و **Momentum** (استفاده از تاریخچه تغییرات برای پایداری).

در ادامه، منطق این روش و دلیل اهمیت آن را بررسی می‌کنیم:

۱. ایده اول: Top-k Selection (انتخاب k برتر)

در هر مرحله از آموزش، مدل هزاران یا میلیون‌ها گرادیان تولید می‌کند. در روش Top-k، ما به جای فرستادن همه آن‌ها، فقط k درصد از بزرگترین گرادیان‌ها (از نظر مقدار مطلق) را انتخاب و ارسال می‌کنیم.

- چرا؟ چون فرض بر این است که گرادیان‌های بزرگ بیشترین تأثیر را در جهت‌دهی به بهینه‌ساز (Optimizer) دارند.
- نتیجه: حجم ترافیک شبکه به شدت (مثلاً ۹۹٪) کاهش می‌یابد.

۲. چالش: چرا Momentum (مومنتوم) اضافه شد؟

اگر فقط Top-k را به تنهایی به کار ببریم، یک مشکل بزرگ پیش می‌آید: **کندی همگرایی**.

گرادیان‌های کوچکی که حذف شده‌اند، ممکن است در بلندمدت جهت درستی را نشان دهند. همچنین در الگوریتم‌های مدرن مثل Adam یا SGD با مومنتوم، ما فقط با "گرادیان لحظه‌ای" کار نمی‌کنیم، بلکه با "میانگین متحرک" آن‌ها کار می‌کنیم.

۳. مکانیزم Top-k با Momentum

در این روش، مومنتوم (سرعت حرکت قبلی) مستقیماً با فرآیند فشرده‌سازی ترکیب می‌شود. دو رویکرد اصلی برای این کار وجود دارد:

الف) مومنتوم محلی (Local Momentum)

در این حالت، هر GPU ابتدا مومنتوم محلی خودش را آپدیت می‌کند و سپس روی خود مقادیر مومنتوم، فیلتر Top-k را اعمال می‌کند. یعنی به جای اینکه بپرسیم "کدام گرادیان بزرگ است؟"، می‌پرسیم "کدام جهت حرکت (مومنتوم) الان قوی‌تر است؟" و فقط آن را ارسال می‌کنیم.

(Error Compensation) (ب) اصلاح خطا با مومنتوم

این همان روشی است که در پاسخ قبلی گفتیم. مقادیری که در Top-k انتخاب نشده‌اند، دور ریخته نمی‌شوند، بلکه در یک بافر (Residual Buffer) ذخیره شده و با مومنتوم مرحله بعد جمع می‌شوند تا در نهایت وقتی به اندازه کافی بزرگ شدند، در لیست Top-k قرار بگیرند و ارسال شوند.

۴. الگوریتم ساده شده (Pseudo-code)

فرآیند در هر GPU به این صورت است:

1. محاسبه گرادیان: g_t (گرادیان مرحله فعلی).
2. به‌روزرسانی بافر خطا: $v_t = v_{t-1} + g_t$ (اضافه کردن گرادیان جدید به مانده‌های قبلی).
3. اعمال مومنتوم (اختیاری): اعمال ضرایب مومنتوم روی v_t .
4. انتخاب k برتر: پیدا کردن بزرگترین مقادیر در v_t و ساختن نسخه فشرده $\text{mask}(v_t)$.
5. ارسال: فرستادن فقط مقادیر انتخاب شده به بقیه GPUها.
6. باقیمانده: مقادیری که ارسال نشدند در v_t می‌مانند تا در مرحله $t+1$ استفاده شوند.

۵. چرا این روش در (Deep Gradient Compression (DGC مهم است؟

این روش قلب تپنده مقاله معروف DGC (سال ۲۰۱۸) است. آن‌ها نشان دادند که با استفاده از **Top-k + Momentum + Error Feedback**، می‌توانند نرخ فشرده‌سازی را به ۱/۶۰۰ برسانند (یعنی ۹۹.۸٪ داده‌ها را نفرستند) بدون اینکه دقت مدل‌های بزرگی مثل ResNet یا BERT حتی ۰.۱٪ افت کند!

مزایای اصلی:

- پایداری: مومنتوم باعث می‌شود جهت حرکت مدل در فضای بهینه‌سازی نوسان نکند.
- کاهش چشمگیر پهنای باند: برای آموزش روی سرورهایی که با اینترنت معمولی یا شبکه ضعیف به هم وصل هستند، حیاتی است.
- جلوگیری از واگرایی: بافر خطا تضمین می‌کند که هیچ دیتایی برای همیشه گم نمی‌شود.

روش powerSGD

روش PowerSGD یکی از روش‌های پیشرفته برای کاهش هزینه‌ی ارتباطی در آموزش مدل‌های بزرگ است که بر پایه تجزیه ماتریسی کم‌رتبه (Low-Rank Matrix Factorization) بنا شده است.

در حالی که روش‌هایی مثل QSGD روی کاهش تعداد بیت‌های هر عدد و SignSGD روی علامت گرادیان تمرکز دارند، PowerSGD از این ایده استفاده می‌کند که گرادیان‌های مدل‌های عمیق معمولاً «کم‌رتبه» هستند؛ یعنی اطلاعات اصلی گرادیان را می‌توان در فضای بسیار کوچکتری خلاصه کرد.

در ادامه توضیح دقیق این روش آمده است:

۱. ایده اصلی: فشرده‌سازی با تقریب کم‌رتبه

فرض کنید گرادیان یکی از لایه‌های مدل شما یک ماتریس بزرگ GG باشد. در روش معمولی، شما باید تمام درایه‌های این ماتریس را ارسال کنید.

اما در PowerSGD، ما فرض می‌کنیم که:

$$GG \approx PQ^{\text{top}}$$

که در آن P و Q ماتریس‌هایی با عرض بسیار کم (مثلاً با رتبه $k=1$ یا $k=2$) هستند. به جای ارسال ماتریس GG ، فقط دو ماتریس کوچک P و Q ارسال می‌شوند که حجم آن‌ها کسری بسیار کوچک از ماتریس اصلی است.

۲. نحوه عملکرد الگوریتم (Power Iteration)

نکته هوشمندانه PowerSGD این است که برای پیدا کردن P و Q ، از الگوریتم سنگین SVD استفاده نمی‌کند (چون خودش خیلی زمان‌بر است). در عوض از روش **Power Iteration** استفاده می‌کند:

- گام اول (محاسبه P): از ماتریس Q گام قبلی استفاده کرده و $P = G \times Q$ را محاسبه می‌کند.
- ارسال و میانگین‌گیری: کارگرها ماتریس کوچک P را ارسال و میانگین می‌گیرند.
- گام دوم (محاسبه Q): با استفاده از P جدید، ماتریس $Q = G^{\text{top}} \times P$ را محاسبه می‌کند.
- ارسال و میانگین‌گیری: ماتریس کوچک Q را ارسال و میانگین گرفته می‌شود.

با این کار، در هر مرحله تخمین ما از فضای اصلی گرادیان دقیق‌تر می‌شود.

۳. ویژگی مهم: استفاده از بازخورد خطا (Error Feedback)

مشابه آنچه در مقاله Karimireddy خواندیم، PowerSGD هم از **Error Feedback** استفاده می‌کند. یعنی تفاوتی که بین گرادیان واقعی (GG) و تقریب کم‌رتبه (PQ^{top}) وجود دارد، دور ریخته نمی‌شود؛ بلکه ذخیره شده و در گام بعدی به گرادیان جدید اضافه می‌شود. این کار باعث می‌شود که حتی با رتبه‌های بسیار پایین (مثل ۱)، دقت مدل حفظ شود.

۴. مقایسه PowerSGD با روش‌های دیگر

ویژگی	QSGD / SignSGD	Local SGD	PowerSGD
نوع فشرده‌سازی	کاهش دقت اعداد (Quantization)	کاهش دفعات ارتباط	کاهش ابعاد ماتریس (Low-rank)
میزان فشرده‌سازی	حدود ۸ تا ۳۲ برابر	متغیر (بسته به H)	بسیار بالا (تا ۱۰۰ برابر یا بیشتر)
سریر محاسباتی	بسیار کم	صفر	متوسط (ضرب ماتریسی اضافه دارد)

مناسب برای	لایه‌های کوچک و بردارهای تخت	شبکه‌های با تأخیر زیاد	لایه‌های کاملاً متصل (Fully Connected) و بزرگ
------------	------------------------------	------------------------	---

۵. چرا PowerSGD محبوب است؟ (مزایای اصلی)

۱. فشردده‌سازی بسیار شدید: برای لایه‌های بزرگ (مثل لایه‌های Transformer یا Embedding)، می‌تواند حجم داده را تا ۹۹٪ کاهش دهد.
۲. ثبات در همگرایی: به دلیل استفاده از Error Feedback و حفظ ساختار ماتریسی، برخلاف کوانتیزه‌سازی شدید، کمتر باعث افت دقت نهایی مدل می‌شود.
۳. بهره‌وری عملیاتی: ضرب ماتریس‌های کوچک در گرادین (که خود یک عملیات GPU-friendly است) بسیار سریع‌تر از روش‌های پیچیده کدگذاری بیت‌ها انجام می‌شود.

مراجع

در اینجا نشان می‌دهیم که هر بخش از پروژه شما به کدامیک از مقالات استخراج‌شده از لیست‌های [Karimireddy et al](#) و Stich ارجاع می‌دهد.

بخش ۱: مبانی و تاریخچه (Foundations & Context)

هدف: ارجاع به مقالاتی که مشکل مقیاس‌پذیری و پایه‌های SGD را مطرح کردند.

- **مبانی SGD:** وقتی از گرادین کاهش تصادفی صحبت می‌کنید، باید به **(Bottou 2010)** ارجاع دهید که تحلیل‌های اولیه همگرایی SGD را برای یادگیری ماشین مقیاس بزرگ ارائه کرد [موجود در لیست [Karimireddy et al](#)].
- **تولید سیستم‌های توزیع‌شده:** برای شروع بحث توزیع‌شده، به **(Dean et al. 2012)** (پروژه Google Brain) و **(Chilimbi et al. 2014)** (پروژه Adam میکروسافت) ارجاع دهید. این‌ها اولین تلاش‌ها برای Parameter Server و Async SGD بودند که نشان می‌دهد چرا ما به روش‌های کارآمد نیاز داریم.
- **انواع مدل‌ها:** وقتی می‌گویید “مدل‌های عمیق”، دقیقاً به **(He et al. 2016a, 2016b)** ارجاع دهید که معماری ResNet را معرفی کردند. این مدل استاندارد طلایی (Benchmark) برای آزمایشات شما در بخش عملی خواهد بود.

بخش ۲: چالش‌های بهینه‌سازی (Optimization Challenges)

هدف: توضیح اینکه چرا آموزش شبکه‌های عصبی سخت است (Non-Convexity).

- **سطوح خطا (Loss Landscapes):** برای توضیح اینکه چرا همگرایی در شبکه‌های عمیق تضمین شده نیست و سطوح خطا پیچیده هستند، به **(Im et al. 2016)** ارجاع دهید.
- **تفاوت SGD و Adam:** وقتی بحث می‌کنید که آیا از SGD استفاده کنیم یا روش‌های تطبیقی (Adaptive)، به **(Kingma & Ba 2014)** (معرفی Adam) و نقد آن در **(Wilson et al. 2017)** (که نشان می‌دهد روش‌های تطبیقی گاهی تعمیم‌پذیری (Generalization) بدتری نسبت به SGD دارند) ارجاع دهید. این توجیه می‌کند که چرا اکثر مقالات فشردده‌سازی (مثل QSGD و SignSGD) روی پایه SGD بنا شده‌اند.

بخش ۳: کوانتیزه‌سازی و فشرده‌سازی گرادینان (Quantization & Compression)

هدف: بررسی روش‌هایی که بیت‌های ارسالی را کم می‌کنند.

- گرادینان‌های سه حالتی: به عنوان مقدمه‌ای بر QSGD و SignSGD، مقاله (Wen et al. 2017) با عنوان *ternGrad* را معرفی کنید. این مقاله گرادینان‌ها را به سه حالت $\{-1, 0, 1\}$ محدود می‌کند و یکی از رقبای اصلی در تحلیل‌های شماس است.
- فشرده‌سازی علامت (Sign-based): اینجا باید مستقیماً به (Bernstein et al. 2018) (مقاله اصلی signSGD) ارجاع دهید. این مقاله نشان داد که گرفتن علامت گرادینان در تنوری می‌تواند کار کند، اما مشکلاتی در همگرایی دارد که بعداً توسط Karimireddy حل شد.
- نسخه‌های بهبود یافته: به (Bernstein et al. 2019) اشاره کنید که نسخه "Majority Vote" را برای تحمل خرابی (Fault Tolerance) اضافه کرد.

بخش ۴: تنک‌سازی (Sparsification)

هدف: ارسال نکردن بخشی از گرادینان‌ها.

- تکنیک‌های اولیه: به (Strom 2015) ارجاع دهید که جزو اولین کسانی بود که پیشنهاد داد فقط گرادینان‌های بزرگتر از یک حد آستانه ارسال شوند (در تشخیص گفتار).
- پراکندگی اتمی: مقاله (Wang et al. 2018) با عنوان *Atomo* را ذکر کنید که روشی برای تجزیه گرادینان به الگوهای تنک (Sparse) است.
- انتشار به عقب تنک: مقاله (Sun et al. 2017) با عنوان *meProp* را ذکر کنید که حتی در مرحله Backpropagation هم از ماتریس‌های تنک استفاده می‌کند تا محاسبات را سریع کند (این مقاله در لیست مراجع Stich بود).

بخش ۵: بازخورد خطا (Error Feedback - EF)

هدف: حل مشکل واگرایی روش‌های فشرده.

- قهرمان بحث: در اینجا ارجاع اصلی شما خود مقاله (Karimireddy et al. 2019) است.
- استدلال: شما با استناد به این مقاله می‌گویید که روش‌های (Bernstein 2018) و (Wen 2017) بدون مکانیزم اصلاح خطا (EF) ممکن است روی داده‌های پیچیده واگرا شوند. Karimireddy ثابت می‌کند که EF برای فشرده‌سازی‌های "اریب" (Biased) ضروری است.

بخش ۶: Local SGD

هدف: کاهش فرکانس ارتباطات به جای حجم بسته‌ها.

- ارجاع اصلی: مقاله (Stich 2019).
- جایگاه: این روش را به عنوان جایگزین کامل برای روش‌های فشرده‌سازی (بخش ۳ و ۴) معرفی کنید، به خصوص زمانی که "تأخیر شبکه" (Latency) بالاست، نه "پهنای باند" (Bandwidth).

بخش ۷ (عملیاتی): تحلیل روش‌ها تحت شرایط مختلف (ماتریس ارجاع)

در این بخش که بخش تحلیل پروژه شماسست، از رفرنس‌ها برای توجیه فرضیات خود استفاده کنید:

دلیل استناد	ارجاع پشتیبان (Justification) (Reference)	روش پیشنهادی	سناریوی شبکه / مدل
برنشتاین نشان داد که ارسال ۱ بیت به جای ۳۲ بیت (۳۲x فشرده‌سازی) ممکن است، و کریمی‌ردی مشکل همگرایی آن را حل کرد.	Bernstein et al. (2018) + Karimireddy (2019)	SignSGD + EF	پهنای باند بسیار کم (Low Bandwidth)
استیچ نشان داد که با HHH گام لوکال، تعداد کل ارتباطات HHH برابر کم می‌شود که مستقیماً با تأخیر مقابله می‌کند.	Stich (2019)	Local SGD	تأخیر بالا (High Latency)
ویلسون نشان داد روش‌های پیچیده (مثل Adam) گاهی به مینیمم‌های تخت (Flat Minima) نمی‌رسند، پس SGD خالص همیشه باید یک خط مبنا (Baseline) باشد.	Wilson et al. (2017)	SGD ((Baseline	دقت مدل و تعمیم‌پذیری (Generalization)
برای اینکه نتایج پروژه معتبر باشد، باید روی معماری استاندارد ResNet که در این مقاله معرفی شده تست شود.	He et al. (2016a)	ResNet-18 /50	معماری مدل آزمایش

مهم ترین مراجع

برای گزارش نهایی، این‌ها مهم‌ترین مراجعی هستند که از فایل‌های شما استخراج شده‌اند و باید در لیست منابع قرار گیرند:

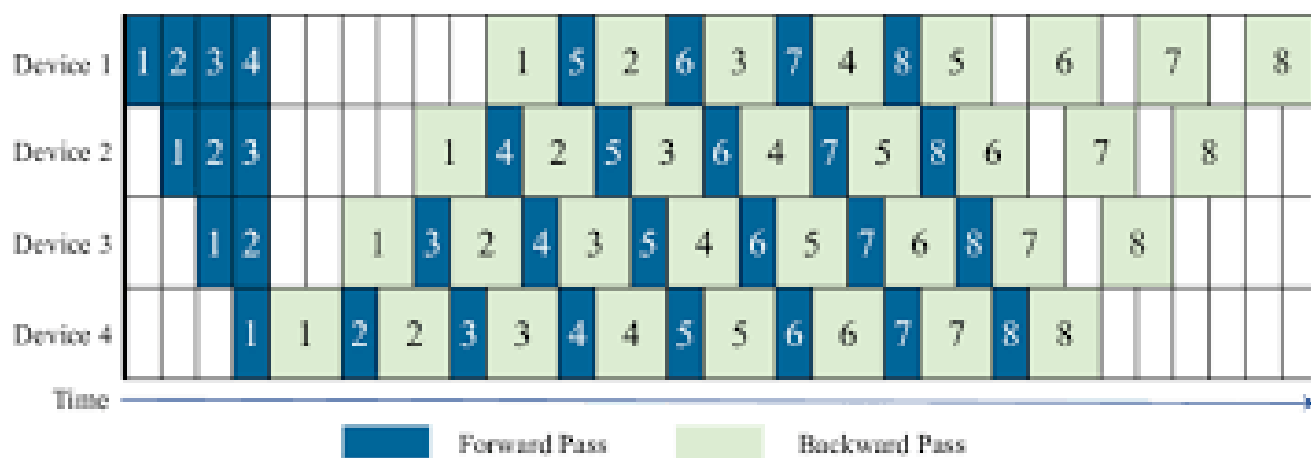
1. Bernstein et al. (2018): *signSGD: compressed optimisation for non-convex problems*, ICML.
2. Stich (2019): *Local SGD Converges Fast and Communicates Little*, ICLR.
3. Karimireddy et al. (2019): *Error Feedback Fixes SignSGD...*, NeurIPS.
4. Alistarh et al. (2017): *QSGD: Communication-Efficient SGD...*, NIPS.
5. Wen et al. (2017): *TernGrad: Ternary gradients...*, NIPS.
6. Dean et al. (2012): *Large Scale Distributed Deep Networks*, NIPS.
7. He et al. (2016): *Deep residual learning for image recognition*, CVPR.

بخش دوم ارائه:

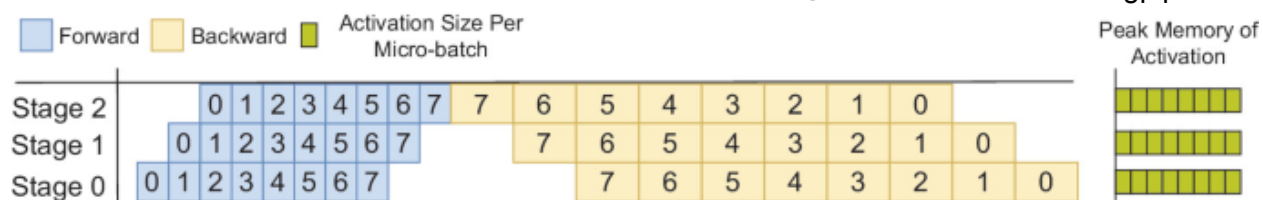
روش gpipe



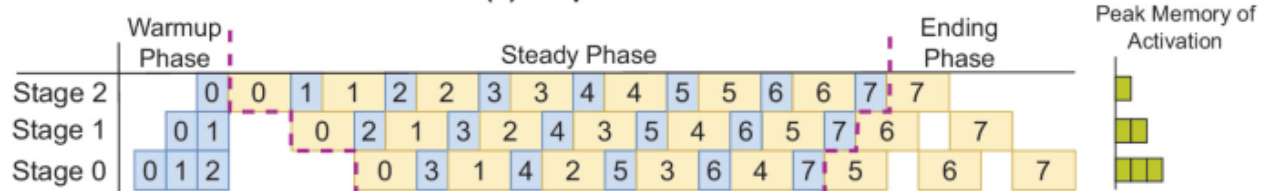
روش 1f1b



مقایسه gpipe و 1f1b از لحاظ حافظه مصرفی



(a) GPipe



(b) 1F1B pipeline

Figure 1 is a Gantt chart illustrating the execution of four devices (Device 1 to Device 4) over time. The chart is divided into four columns representing different tasks: F (blue), B (teal), W (green), and Optimizer step (yellow). The time axis is labeled 'Time' and 't'. The chart shows that Device 1 and Device 2 execute tasks F, B, and W, while Device 3 and Device 4 execute tasks F, B, and W. The Optimizer step is shown as a yellow block at the end of the timeline for each device.

[illegible]

7P1D Forward Backward Optimizer step

Device	1	2	3	4				1	1	5	2	2	6	3	3	7	4	4	8	5	5	6	6	7	7	8	8			1	2	3	4				1	1	5	
Device 1	1	2	3	4				1	1	5	2	2	6	3	3	7	4	4	8	5	5	6	6	7	7	8	8			1	2	3	4				1	1	5	
Device 2		1	2	3				1	4	2	1	5	3	2	6	4	3	7	5	4	8	6	5	7	6	8	7	8			1	2	3				1	4	2	1
Device 3			1	2				1	3	2	4	3	1	5	4	2	6	5	3	7	6	4	8	7	5	8	6	7	8			1	2			1	3	2	4	3
Device 4				1	1	2	2	3	3	4	4	1	5	5	2	6	6	3	7	7	4	8	8	5	6	7	8					1	1	2	2	3	3	4		

Time →

Legend: ■ F ■ B ■ W ■ Optimizer step

DualPipe

Time →

Legend: F (Forward), B (Backward), W (Backward for input), Optimizer step

Device 0: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884 885 886 887 888 889 890 891 892 893 894 895 896 897 898 899 900 901 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918 919 920 921 922 923 924 925 926 927 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 944 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 968 969 970 971 972 973 974 975 976 977 978 979 980 981 982 983 984 985 986 987 988 989 990 991 992 993 994 995 996 997 998 999 1000

Time →

Legend: Forward, Backward, Backward for input, Backward for weights, Overlapped forward & Backward

چرا backward دو برابر forward در gpipe و 1f1b است؟

To understand why the backward pass is usually $2\times$ the duration of the forward pass—and how Zero Bubble "cheats" this math—we have to look at what's actually happening inside the GPU's matrix multiplication (GEMM) engines.

1. The "Three-GEMM" Rule

In deep learning, we calculate three main things during a training step. Think of the Forward pass as **one** operation and the Backward pass as **two** separate operations.

- **Forward Pass (F):** Calculates the output Y using input X and weights W .
 - *Math:* $Y=W \cdot X$
 - **Cost: 1 Unit of Time**
- **Backward Pass Part 1 (∇X):** Calculates the gradient of the input. This is what the **previous** layer (Layer $n-1$) is waiting for so it can start its work.
 - *Math:* $\nabla X=W^T \cdot \nabla Y$
 - **Cost: 1 Unit of Time**
- **Backward Pass Part 2 (∇W):** Calculates the gradient of the weights. This is only used by the optimizer to update the model later. **No other layer needs this to proceed.**
 - *Math:* $\nabla W=\nabla Y \cdot X^T$
 - **Cost: 1 Unit of Time**

2. Why GPipe and 1F1B take $2\times$ Forward

In standard libraries (like PyTorch or early Megatron-LM), these two backward steps (∇X and ∇W) are bundled together into one single "Backward" block.

When Layer n starts its backward pass, it calculates **both** ∇X and ∇W before it considers itself "done."

$$\text{Total Backward Time} = \text{Time}(\nabla X) + \text{Time}(\nabla W) \approx 2 \times \text{Time}(F)$$

Because Layer $n-1$ is sitting idle waiting for ∇X , it has to wait through the calculation of ∇W as well, even though ∇W is technically "useless" for the pipeline's flow. This creates the massive "bubbles" (idle time) you see in standard pipeline diagrams.

3. The "Zero Bubble" Magic: Splitting the Atom

The core "aha!" moment of the **Zero Bubble** paper is that we can split the backward pass into two distinct tasks: **B** (the activation gradient ∇X) and **W** (the weight gradient ∇W).

In Zero Bubble, the system recognizes that:

1. **Task B** is high-priority (it unblocks the next guy).
2. **Task W** is low-priority (it just needs to be done eventually).

The Mathematical Shift:

In Zero Bubble, we don't treat the backward pass as a $2F$ block. We treat it as two $1F$ blocks.

- **The Pipeline Flow:** Layer n calculates **Task B** (∇X) and immediately sends it to Layer $n-1$. To the rest of the pipeline, it looks like the backward pass only took $1 \times$ Forward time.
- **Filling the Gaps:** While Layer $n-1$ is busy with the data it just received, Layer n stays busy by calculating **Task W** (∇W) in the "bubbles" (the gaps) where it would normally be doing nothing.

Comparison Summary

Feature	GPipe / 1F1B	Zero Bubble
Backward Unit	Atomic ($B+W$)	Split (B and W separate)
Blocking Time	$2 \times$ Forward	$1 \times$ Forward

Idle Time (Bubbles)	Massive (Waiting for $B+W$)	Zero (Bubbles are filled with W)
Math Accuracy	Exact	Exact

Why this works:

Mathematically, the total work is still $3 \times \text{Forward}$ ($1F+1B+1W$). Zero Bubble doesn't delete the work; it just rearranges the $1W$ part so it happens during "recess" (the bubble) instead of during "class time" (the critical path).

Would you like me to show you the specific schedule diagram that illustrates how these W tasks are tucked into the pipeline gaps?

خب من از این چت کمک گرفتم برای توضیح ارائه ام:

<https://gapgpt.app/share/55d97fc2-22e9-481c-aa6c-e59e05741edb>

<https://gemini.google.com/share/d4b65438f68d>

<https://gemini.google.com/share/8e85a2574963>

<https://gemini.google.com/share/7f8329979e29>

<https://gemini.google.com/share/b63706deaa35>

لینک عنوان مقاله ها را در ارائه قرار دادم. با تشکر